

107. Au pixel près : le lutin glisse

Une lettre saute de case en case, 8 pixels d'un coup ; un lutin avance d'un seul pixel.

1. Ce qu'il faut comprendre

Jusqu'ici, presque tout bougeait **case par case** : `texte()` et `poser()` écrivent dans la grille du fond, 20 colonnes sur 18 lignes. Une case fait **8 × 8 pixels** : passer d'une case à la suivante, c'est un saut de 8 pixels. À l'œil, ça **saute**.

Un **lutin** (chapitre 7) ne vit pas dans la grille. `sprite(0, x, y, HEROS)` le pose **au pixel près** : `x` va de 0 (tout à gauche) à 152 (tout à droite : 160 pixels de large, moins les 8 du lutin), `y` de 0 à 136. Ajouter 1 à `x`, c'est avancer d'un **seul pixel** : le mouvement devient doux.

Ce qui est nouveau ici : une seule variable, deux façons de la montrer. `x` compte en pixels. Le lutin est posé à `x`. Le O, lui, est écrit à la colonne `x / 8` : la division entière du chapitre 1 range le pixel dans sa case. De 0 à 7 → case 0 ; de 8 à 15 → case 1 ; 76 / 8 = 9 (reste 4) → case 9.

Pousse DROITE et regarde : le lutin glisse, et le O ne bouge qu'un **pas sur huit**, d'un bloc. C'est toute la différence entre un jeu de cases (le snake, un jeu de plateau) et un jeu d'action (Mario, Zelda).

`case_o` retient la colonne où le O est écrit. On ne réécrit le O **que quand** `x / 8` **change** : `texte()` attend une pause de la console (chapitre 3), et le lutin, lui, n'attend rien. Réécrire le O à chaque image, ce serait du travail pour rien.

À toi : ajoute HAUT et BAS, avec `y`, et fais suivre le O sur les lignes avec `y / 8`.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <sprite>` → [0.69.4. La fonction sprite\(\) — un lutin au pixel près](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **807 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	65 o	← la console, à l'allumage — la mise en route de la console
main()	176 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte — attend de pouvoir écrire une case
AdresseCase	10 o	← texte() — trouve l'adresse d'une case de l'écran
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
RangerLutins	11 o	← démarrage — range les 40 lutins hors de l'écran

```

InstallerTransfert  15 o ← démarrage – installe la copie des lutins en mémoire rapide
CopierTuiles        130 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte        14 o ← démarrage – vide les deux cartes de l'écran
LireManette         37 o ← bouton() – lit les huit boutons
Tuiles              224 o ← CopierTuiles – 13 lettres sur 43, et 1 tuile dessinée
DonneesTransfert    10 o ← InstallerTransfert – la petite routine qui copie les lutins
"EN CASES : 8 PI..." 19 o ← texte() – un texte gravé
"EN PIXELS : 1 P..." 19 o ← texte() – un texte gravé
"0"                 1 o ← texte() – un texte gravé
" "                 1 o ← texte() – un texte gravé
"0"                 1 o ← texte() – un texte gravé

```

Lettres gravées : ACEILNOPSX18:

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, Notes

Fonctions du langage appelées :

```

texte() × 5      76 o sur place, dans main() + AdresseCase (10 o), EcrireTexte (10 o)
sprite() × 1     23 o sur place, dans main()
bouton() × 2     10 o sur place, dans main() + LireManette (37 o)
image() × 1      6 o sur place, dans main() + AttendreImage (28 o)

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// CE PROGRAMME : GAUCHE / DROITE déplacent un héros AU PIXEL près (il glisse),
// et un 0 qui, lui, avance CASE par case (il saute de 8 pixels d'un coup).
//
// Rappel : l'écran fait 160 × 144 pixels, soit 20 × 18 cases de 8 × 8.
// Ce qui est nouveau : une seule variable x (en pixels), montrée de deux façons :
//   le lutin est posé à x ; le 0 est écrit à la colonne x / 8.

// Le héros : un lutin de 8 × 8 pixels.
// Les points sont l'indice 0 : chez un lutin, ils sont TRANSPARENTS.

#include <Tuile>    // un dessin de 8 × 8 pixels
#include <texte>    // écrit un texte à l'écran
#include <bouton>   // lit un bouton de la manette
#include <sprite>   // place un lutin de 8 × 8 au pixel près

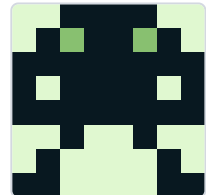
```

2. UN DESSIN — HEROS

```

Tuile HEROS = {
    "..####..",
    ".#-##-#.",
    "#####",
    "#.####.#",
    "#####",
    "..#..#..",
    ".#...#..",
    "##...##",
};

```



à l'écran

3. UNE VARIABLE — X

```

uint8_t x = 76;    // la position du lutin, EN PIXELS : 0 à gauche ... 152 à droite
                  // (160 pixels de large, moins les 8 du lutin)

```

4. UNE VARIABLE — CASE_O

```

uint8_t case_o = 9; // la colonne où le 0 est écrit en ce moment : 76 / 8 = 9

```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    texte(1, 1, "EN CASES : 8 PIXELS");
    texte(1, 7, "EN PIXELS : 1 PIXEL");
    texte(case_o, 4, "0");                  // le 0, dans la grille du fond (ligne 4)

    while (true) {                          // La boucle du jeu :
        image();                            // une image : 60 par seconde

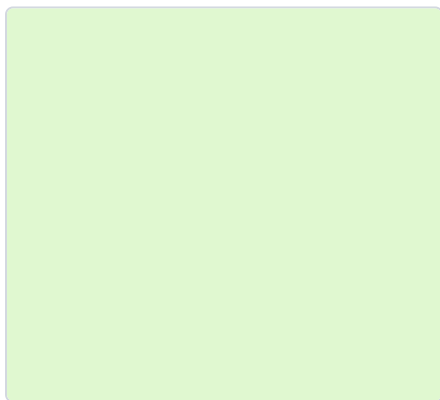
        // Le pixel : un de plus, ou un de moins, à chaque image.
        if (bouton(DROITE) && x < 152) x++;
        if (bouton(GAUCHE) && x > 0) x--;

        // Le lutin suit x au pixel près (ligne de pixels 80 : sous le texte).
        sprite(0, x, 80, HEROS);

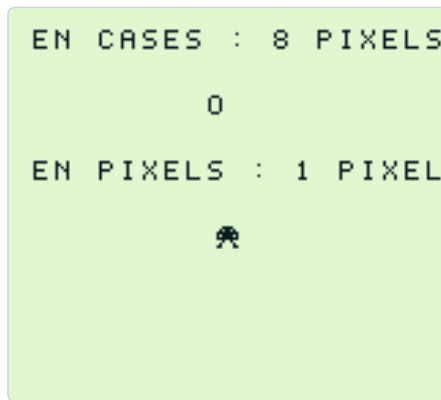
        // La case : x / 8 (division entière). Le 0 ne change de place que quand elle change.
        // x de 72 à 79 → case 9 ; x de 80 à 87 → case 10 ; etc.
        if (x / 8 != case_o) {
            texte(case_o, 4, " ");          // on efface l'ancien 0
            case_o = x / 8;                 // la nouvelle case : 77 / 8 = 9, 80 / 8 = 10 ...
            texte(case_o, 4, "0");          // et on l'écrit là
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



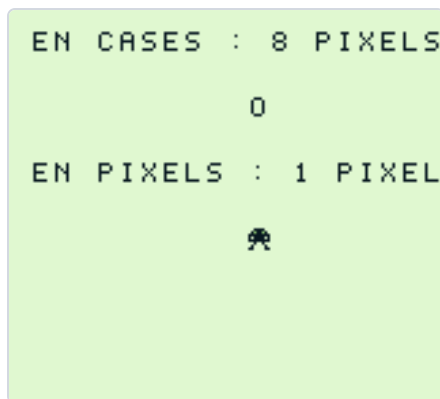
la toute première image — la console vient de démarrer — image 1



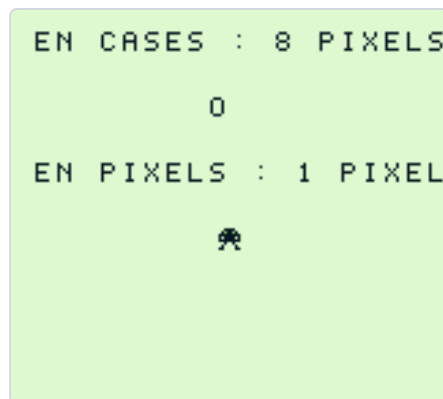
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



DROITE — l'écran a changé



GAUCHE — l'écran a changé

5. Ce qu'on doit voir

Un O dans la grille, un petit héros en dessous. DROITE et GAUCHE : le héros glisse pixel par pixel, le O saute de case en case.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le lutin part du pixel 76
- ✓ il avance d'un pixel à la fois (76 puis 100)
- ✓ le lutin est bien à x
- ✓ le O est dans la case $x / 8$ (case 12, x 100)

Ensuite : **108. La vitesse : un pas toutes les N images** — chapitre 17 : Le mouvement : les lutins au pixel près. · gameboy3 — écrire une cartouche Game Boy en C++